

Fondamenti di Computer Graphics (modulo 2)

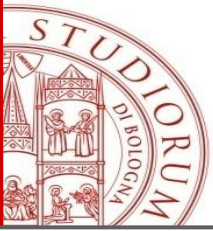
ACADEMIC YEAR 2025/2026

Learning Three.js

Serena Morigi

Office: F2, 4th floor

serena.morigi@unibo.it



Tools you need to work with Three.js

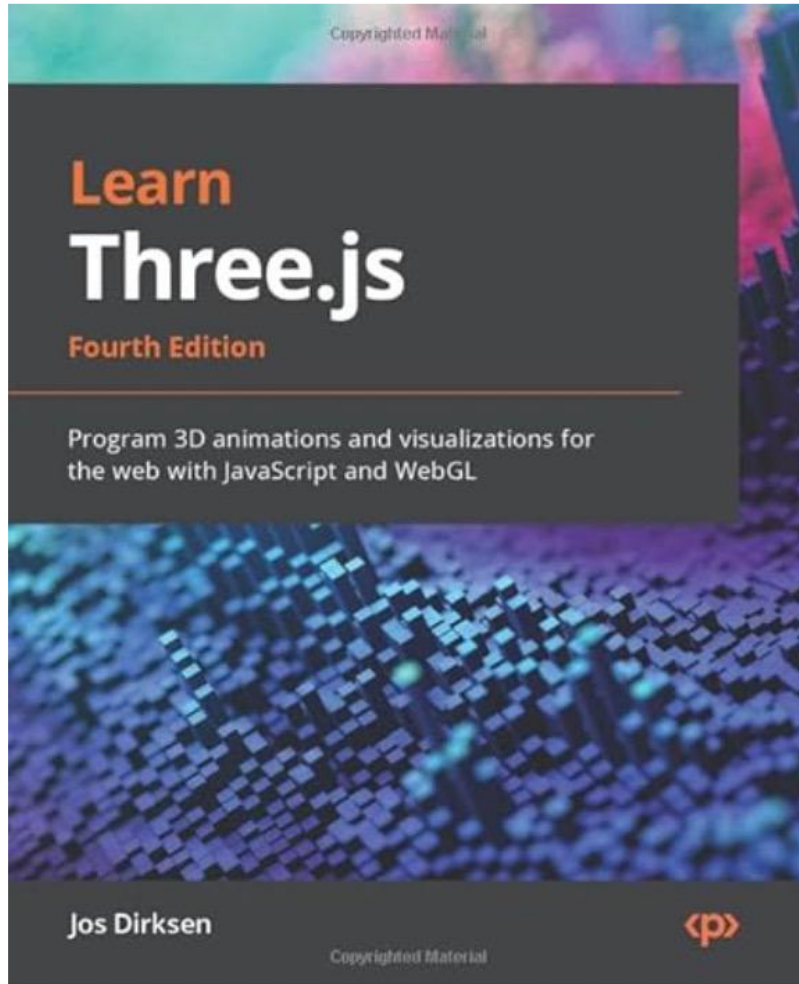
- To create Three.js WebGL applications you need a **text editor** and one of the supported **browsers** to render the results:
 - **Visual Studio Code**
 - **Firefox** (Chrome,..)
- **Download Three.js**

Con il download viene scaricato il file three.js-master.zip che contiene l'intera distribuzione e che permetterà di utilizzare la libreria three.js, ma anche tanti plug-in, in locale.

Get the **server** up and running

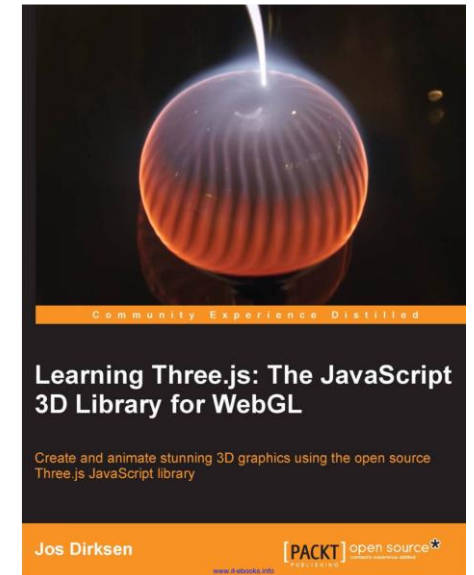
- `$ npm run serve`
- LiveServer

Textbook

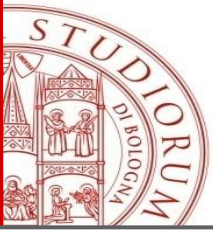


Only the **first edition** is available for free download from the internet - and you can also find it among the course documents - whereas subsequent editions are not.

We will therefore use the first edition as a guide, but we will utilize the **code/examples from the fourth edition**; these code samples use `three.module.js`



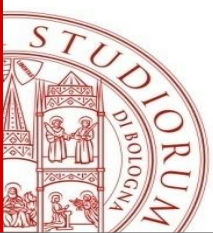
Textbook : “Learning Three.js”, Jos Dirksen, fourth edition
You can download the example code files for this book from GitHub at <https://github.com/PacktPublishing/Learn-Three.js-Fourth-edition>



Chapter 01

Creating Your First 3D Scene with Three.js

In this chapter, you will create your first 3D scene with **Three.js** and walk through the basic steps required to get started.
You will also be able to animate your first 3D scene and manage a control panel useful for debugging.



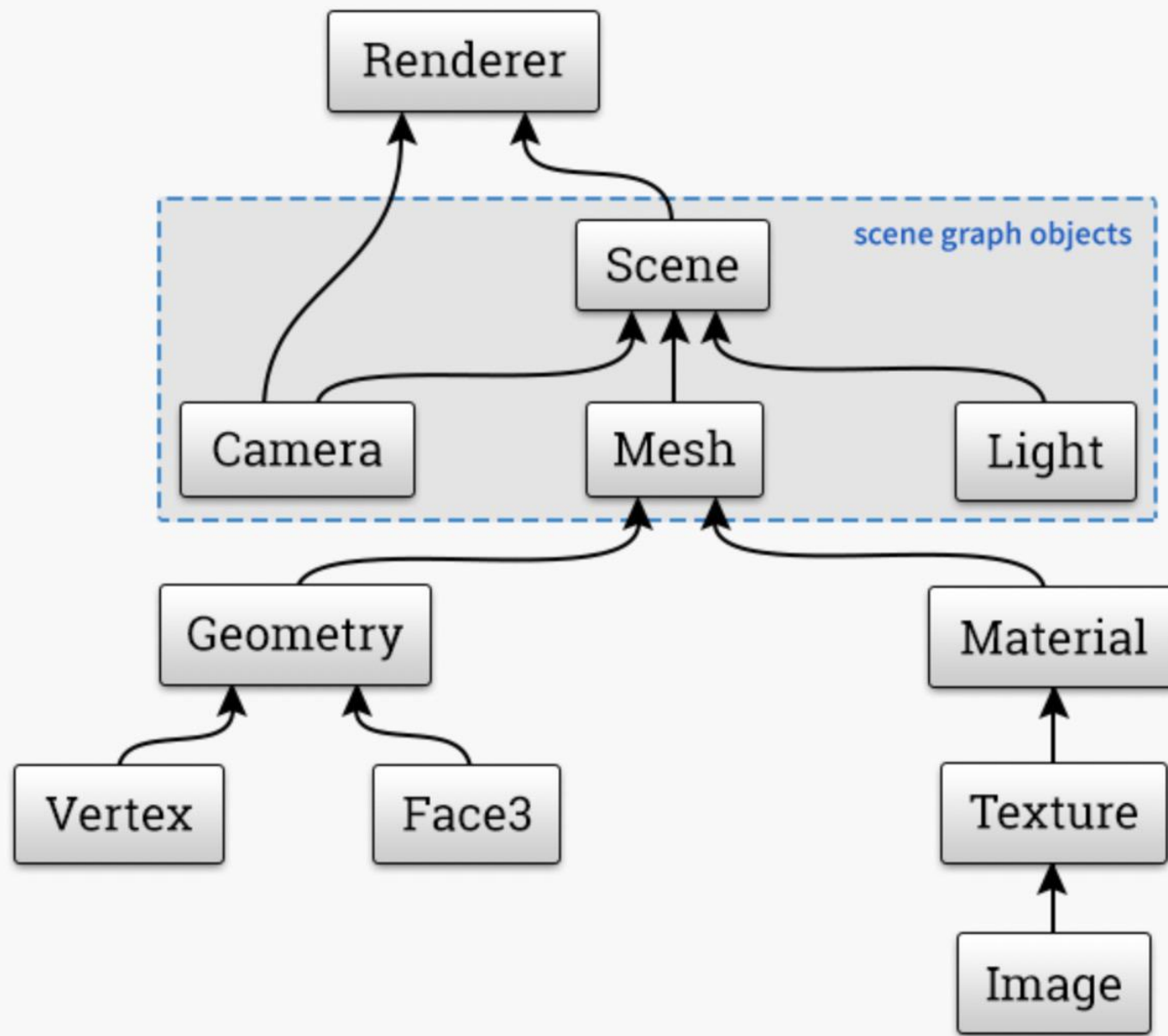
Each Three.js application at least needs a **scene**, and a **renderer**.

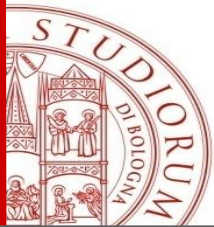
The **scene** is the container that holds all the objects (meshes, cameras, and lights)

The **renderer** takes care of creating the output on the screen

THREE.Scene is a structure that is called a scene graph
a scene graph consists of a set of nodes in a tree structure.

Scene Graph



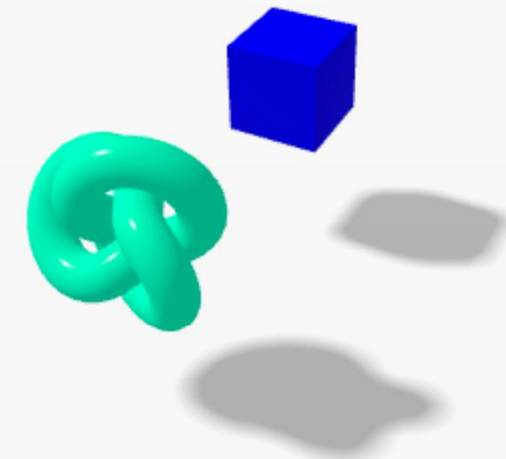


chapter-1/getting-started.js

```
import * as THREE from "three";
import Stats from 'three/examples/jsm/libs/stats.module'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls'
// create a scene
...
// setup camera
...
// setup the renderer and attach to canvas
...
// add lights
...
// create a cube and torus knot and add them to
...
// create a very large ground plane
...
// add orbitcontrols to pan around the scene using the
  mouse
...
// add statistics to monitor the framerate
...
// render the scene
```

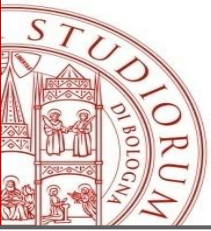
```
// setup the renderer and attach to canvas
renderer = new THREE.WebGLRenderer( { antialias: true } );
renderer.setSize( window.innerWidth, window.innerHeight );
document.body.appendChild( renderer.domElement );
```

This last step adds an HTML canvas element to the page, which shows the output of the renderer



OrbitControls: This will allow you to use the mouse to rotate and pan around the scene

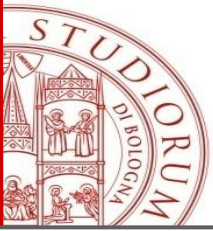
Lights: This allows us to use some more advanced materials, cast shadows, and generally make our scene look better



chapter-1/getting-started.js : Mesh

```
...  
scene.add(cube) ;  
...  
scene.add(torusKnotMesh) ;  
...  
scene.add(groundMesh) ;
```

- We create the shape `torusKnotGeometry` – that is, the geometry of the objects: a `THREE.BoxGeometry`, a `THREE.TorusKnotBufferGeometry`, and a `THREE.PlaneBufferGeometry`
- We create the material `torusKnotMat`
- We tell Three.js that the cube and the torus knot cast shadows and that the ground will receive shadows.
- Finally, from the shape and the material, we create a `THREE.Mesh`, position the mesh, and add it to the scene.
- call `renderer.render(scene, camera)` . You will see the result on your screen



Adding a render (an animation) loop

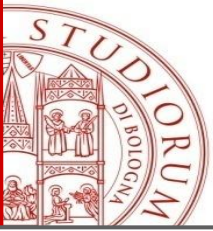
```
function animate() {  
    requestAnimationFrame(animate);  
    renderer.render(scene, camera);  
} stats.update(); // update FPS  
animate();
```

With `requestAnimationFrame`, we don't tell the browser when it needs to update the screen; we ask the browser to run the supplied function when it's most opportune. Usually, this results in a frame rate of about 60 or 120 FPS (depending on your hardware). You do any drawing you need to do in the supplied function, and the browser will make sure it is painted as smoothly and efficiently as possible.

```
import Stats from 'three/examples/jsm/libs/stats.module'  
const stats = Stats()  
document.body.appendChild(stats.dom)
```

library called **stats.js**, which gives us information about the frame rate the animation is running at. This library, from the same author as `Three.js`, renders a small graph that shows us information about the rate at which the scene is rendered.

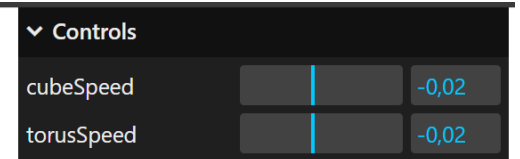
Animating the meshes



Using lil-gui to control properties

use the **lil-gui library** from <https://lil-gui.georgealways.com/>.

This library allows us to quickly create a simple control UI

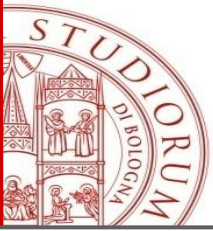


```
import GUI from "lil-gui";
...
const gui = new GUI();
const props = {
  cubeSpeed: 0.01,
  torusSpeed: 0.01,
};
gui.add(props, 'cubeSpeed', -0.2, 0.2, 0.01)
gui.add(props, 'torusSpeed', -0.2, 0.2, 0.01)
function animate() {
  ...
  cube.rotation.x += props.cubeSpeed;
  cube.rotation.y += props.cubeSpeed;
  cube.rotation.z += props.cubeSpeed;
  torusKnotMesh.rotation.x -= props.torusSpeed;
  torusKnotMesh.rotation.y += props.torusSpeed;
  torusKnotMesh.rotation.z -= props.torusSpeed;
  ...
}
```

We create a new control element (new GUI) and configured two controls: cubeSpeed and torusSpeed.

In each animation step, we'll just look up the current values and use those to rotate the meshes.



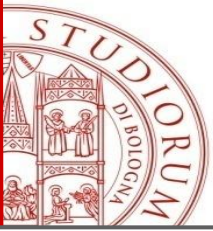


Summary

In chapter 01, we created a **THREE.Scene()** object and looked at the basics of the Three.js library. For a scene to be rendered, we need three types of components:

- **Camera:** determines what is displayed on the screen.
- **Lights:** have an effect on how materials are shown and used, or in the rendering of shadows.
- **Objects:** the main objects that are rendered while observing them from the camera: cubes, spheres, etc.

The **THREE.Scene()** object acts as a container for these various components.

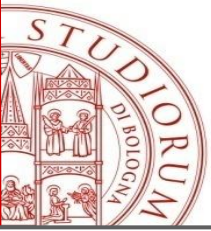


Chapter 02

Working with the Basic Components that Make Up a Three.js Scene

In this chapter, we will cover the following topics:

- Creating a scene
- How geometries and meshes are related
- Using different cameras for different scenes



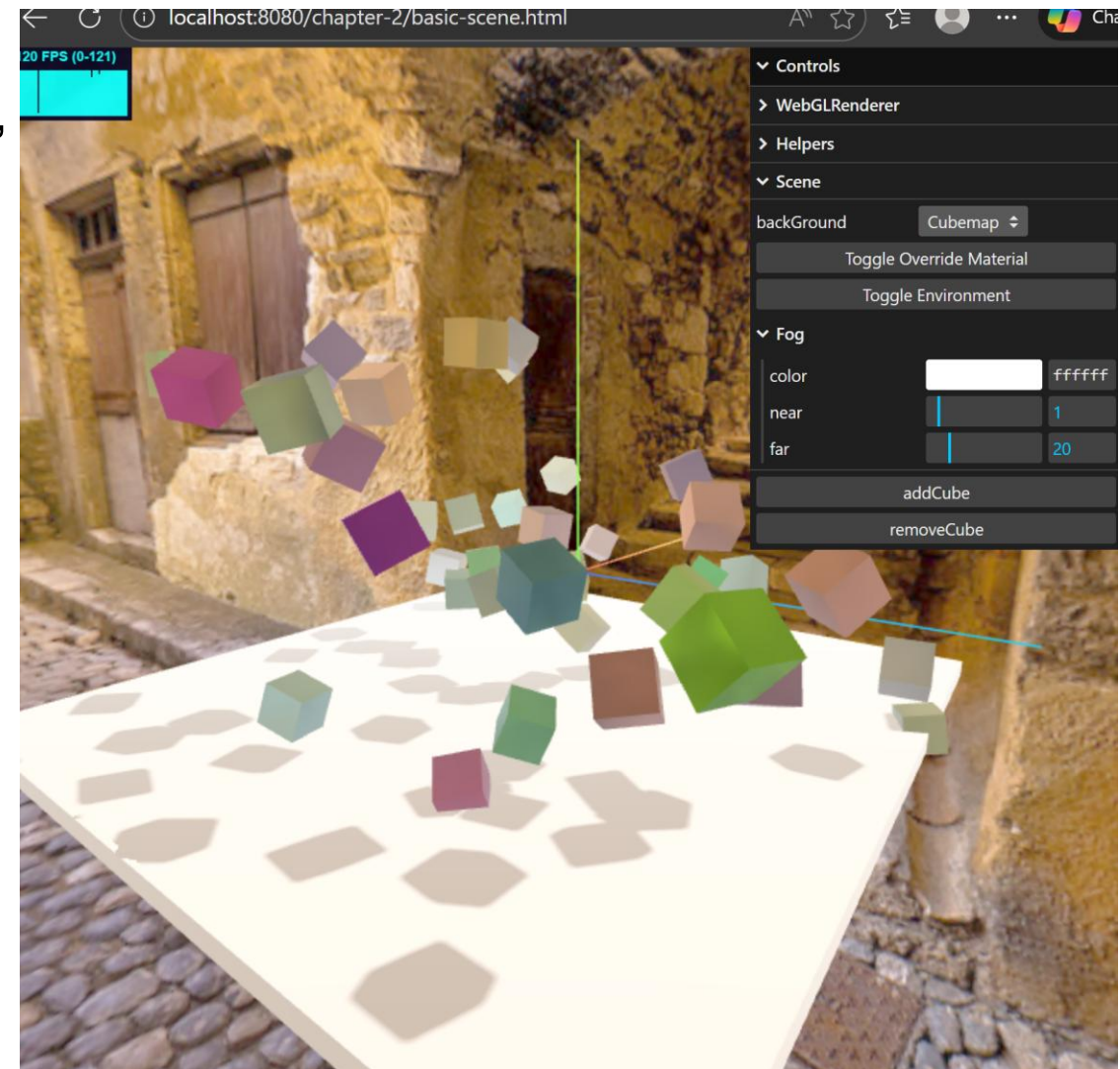
chapter-2/basic-scene.js

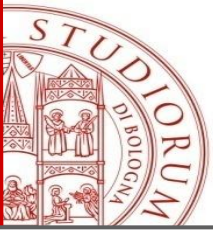
We can use (by import) code from `bootstrap/bootstrap.js`, `bootstrap/floor.js`, and `bootstrap/lighting.js` since this is a generic scene setup. These files will

```
// create a camera  
// create a renderer  
// create a scene  
// create the lights  
// create the floor
```

See `control/add-remove-cube-control.js` for adding and removing cubes to the scene

See `controls/scene-controls.js` for how to configure background (solid/texture/environment) as a simple background color for the scene

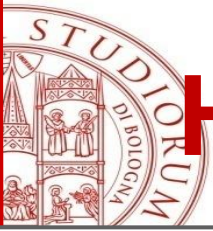




THREE.Scene functions

Helpful functions for the THREE.Scene, related to working with the children of the scene:

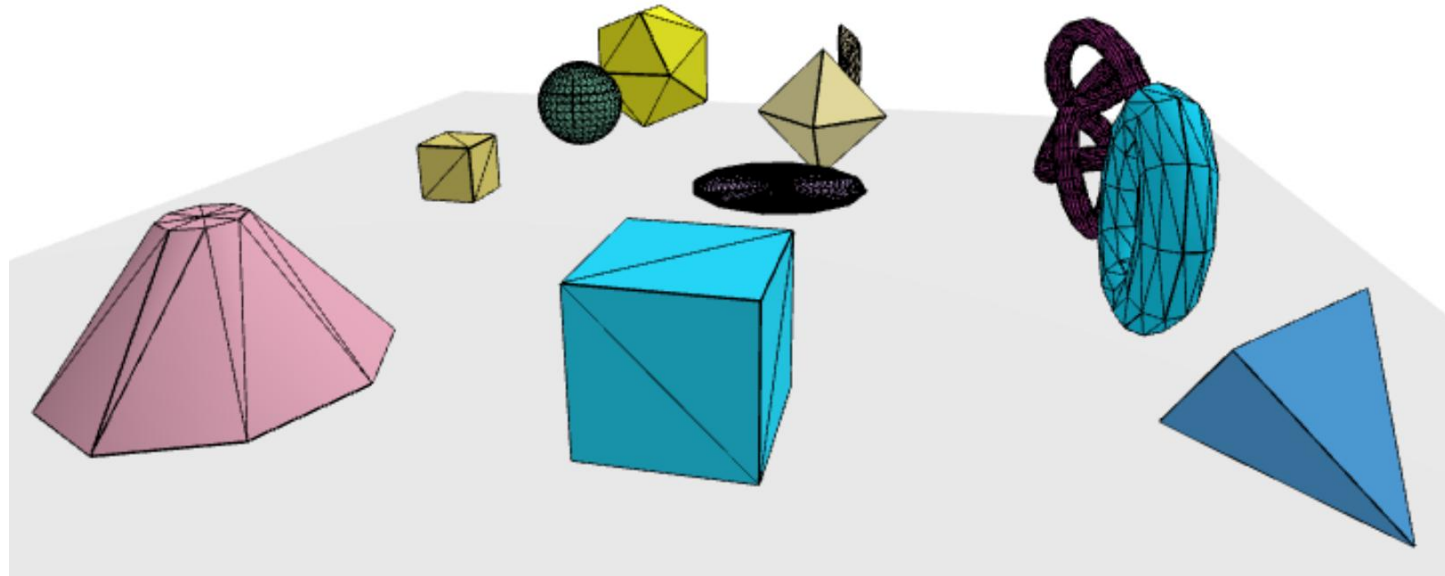
- **add**: which adds the provided object to the scene. If it was previously added to a different THREE.Object3D, it'll be removed from that object.
- **attach**: similar to add, but if you use it, any rotations or translations applied to this object will be kept.
- **getObjectById**: When you add an object to a scene, it gets an ID. The first one gets 1, the second one 2, and so on. With this function, you can get a child based on this ID.
- **getObjectByName**: This returns an object based on its name property. The name is something you can set on an object – this is in contrast with the id property, which is assigned by Three.js.
- **remove**: removes an object from the scene.
- **clear**: removes all the children from the scene.



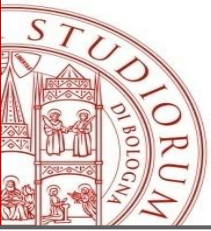
How geometries and meshes are related

- standard geometries available in Three.js:

chapter-2/geometries.js example



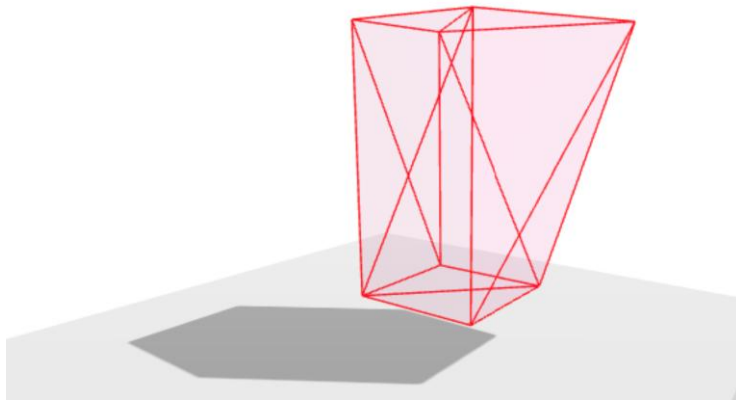
You can either create geometries from scratch using a `THREE.BufferGeometry()` object or use any of the built-in geometries Three.js provides.



Create a geometry from scratch

This code shows how to
create a simple cube

chapter-2/custom-geometry.js example



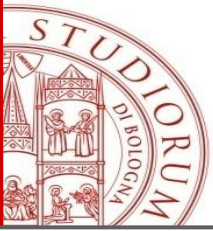
Three.js determines the normal vector for each of the vertices and for the faces.

```
const v = [
  [1, 3, 1],
  [1, 3, -1],
  [1, -1, 1],
  [1, -1, -1],
  [-1, 3, -1],
  [-1, 3, 1],
  [-1, -1, -1],
  [-1, -1, 1]]

const faces = new Float32Array([
  ...v[0], ...v[2], ...v[1],
  ...v[2], ...v[3], ...v[1],
  ...v[4], ...v[6], ...v[5],
  ...v[6], ...v[7], ...v[5],
  ...v[4], ...v[5], ...v[1],
  ...v[5], ...v[0], ...v[1],
  ...v[7], ...v[6], ...v[2],
  ...v[6], ...v[3], ...v[2],
  ...v[5], ...v[7], ...v[0],
  ...v[7], ...v[2], ...v[0],
  ...v[1], ...v[3], ...v[4],
  ...v[3], ...v[6], ...v[4]]);

const bufferGeometry = new THREE.BufferGeometry();
bufferGeometry.setAttribute("position", new THREE.BufferAttribute(faces, 3));
bufferGeometry.computeVertexNormals();
```

sorted for front-facing face

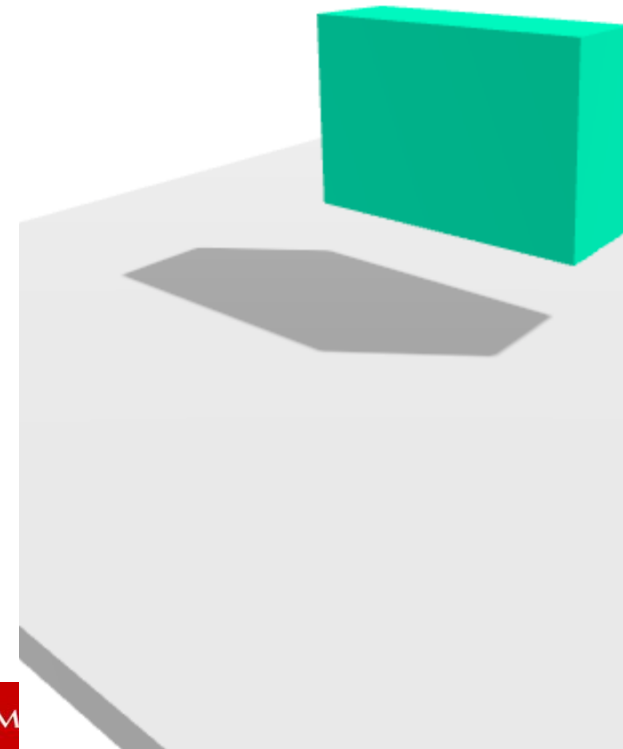


Set properties and functions on meshes

- **position:** This determines the position of the object relative to the position of its parent. Most often, the parent of an object is a `THREE.Scene` object or a `THREE.Group` object.
- **rotation:** you can set the rotation of an object around any of its own axes. Three.js also provides specific functions for rotations around a single axis: `rotateX()`, `rotateY()`, and `rotateZ()`.
- **scale:** This property allows you to scale the object around its x-, y-, and z-axes.
- **translateX() / translateY() and translateZ():** This property moves the object by a specified amount along the corresponding axis.
- **lookAt():** This property points the object to a specific vector in space. This is an alternative to setting the rotation manually.
- **visible:** This property determines whether this mesh should be rendered or not.
- **castShadow:** This property determines whether this mesh casts shadows when it is hit by light. By default, meshes don't cast shadows.

Set properties and functions on meshes

- `chapter-2/mesh-properties.js`
you get a drop-down menu where you can alter all the properties and directly see the result



Controls

- WebGLRenderer
- Helpers
- Scene
- Mesh

scaleX		1
scaleY		1
scaleZ		1
rotationX		0
rotationY		0
rotationZ		0
positionX		0
positionY		0
positionZ		0
translateX		0
translateY		0
translateZ		0

translate

lookAtX		0
lookAtY		0
lookAtZ		0

lookAt

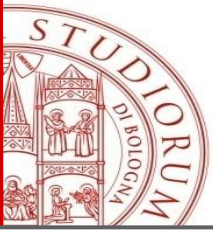
visible ☒

castShadow ☒

rotateOnWorldAxisX

rotateOnWorldAxisY

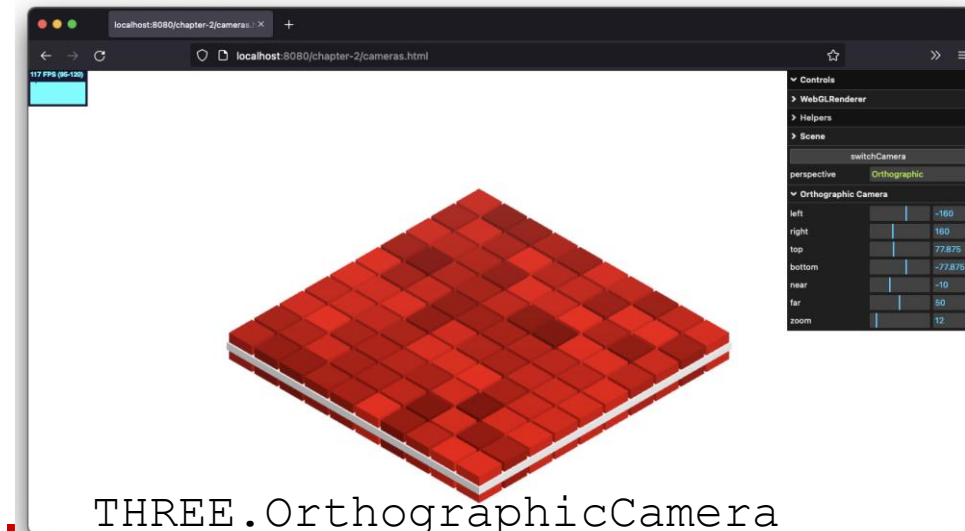
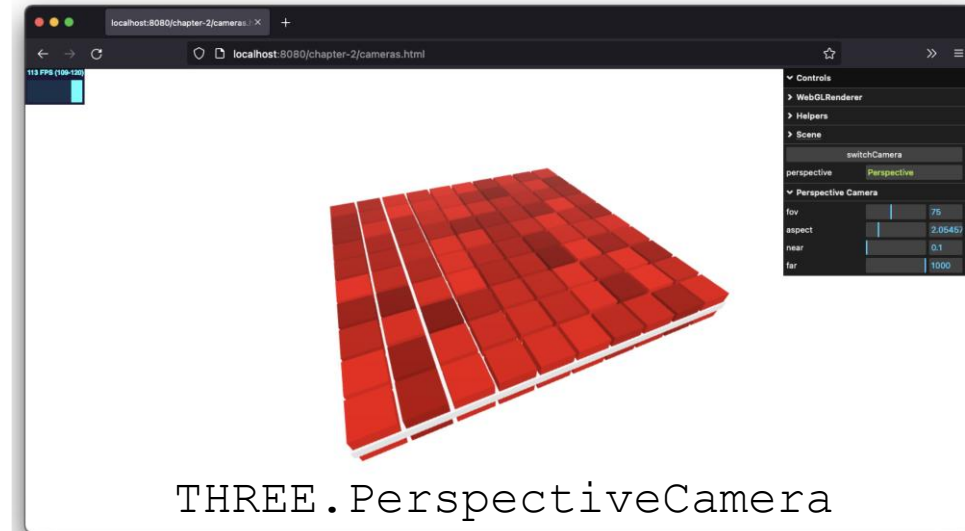
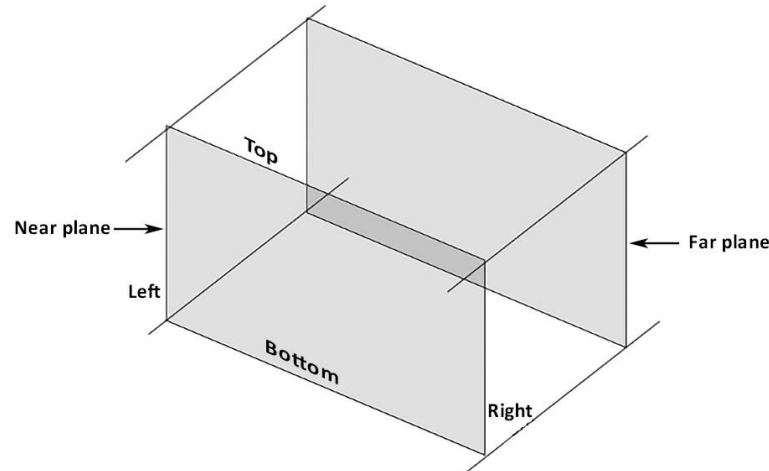
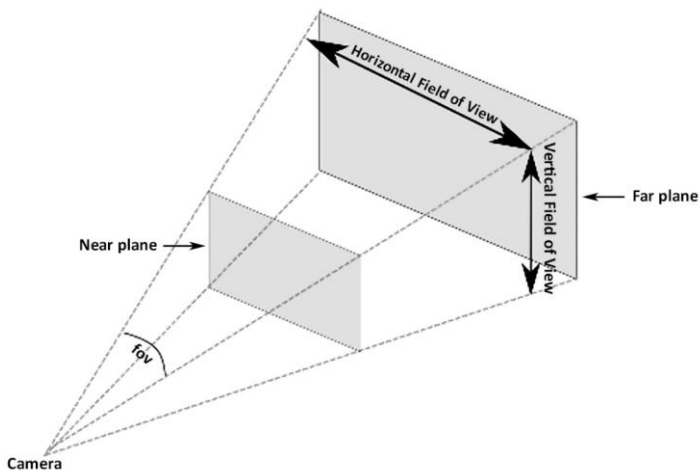
rotateOnWorldAxisZ

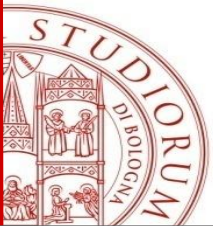


The orthographic camera versus the perspective camera

- There are two different camera types in Three.js: the orthographic camera and the perspective camera.
- Three.js also provides a couple of very specific cameras for creating scenes that can be viewed using 3D glasses or VR gear.

chapter2/cameras.js



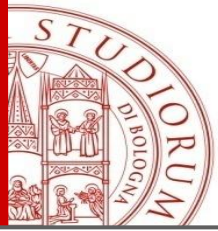


Chapter 3

Working with the Different Light Sources Available in Three.js

In this chapter, we will cover the following topics:

- Different kinds of lighting in Three.js
- Working with basic lights
- Working with special lights



What lighting types are provided in Three.js?

- **THREE.AmbientLight:** This is a basic light, the color of which is added to the current color of the objects in the scene.
- **THREE.PointLight:** This is a single point in space from which light spreads in all directions.
- **THREE.SpotLight:** This light source has a cone-like effect like that of a desk lamp, a spotlight in the ceiling, or a torch. This light can cast shadows.
- **THREE.DirectionalLight:** This is also called infinite light. The light rays from this light can be seen as parallel, similar to the light from the Sun. This light can also be used to create shadows.
- **THREE.HemisphereLight:** This is a special light and can be used to create more natural-looking outdoor lighting by simulating a reflective surface and a faintly illuminating sky.
- **THREE.RectAreaLight:** instead of a single point in space, you can specify an area from which light emanates. THREE.RectAreaLight doesn't cast any shadows.
- **THREE.LightProbe:** This is a special kind of light source where, based on the environment map used, a dynamic ambient light source is created to light the scene.
- **THREE.LensFlare:** This is not a light source, but with THREE.LensFlare, you can add a lens flare effect to the lights in your scene.

Examples in chapter 3:

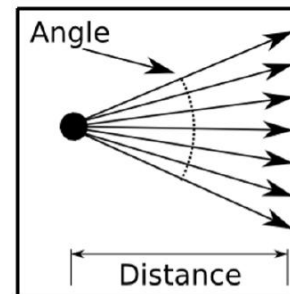
How to create a THREE.AmbientLight:

```
const color = new THREE.Color(0xffffff);
const light = new THREE.AmbientLight(color);
scene.add(light);
```

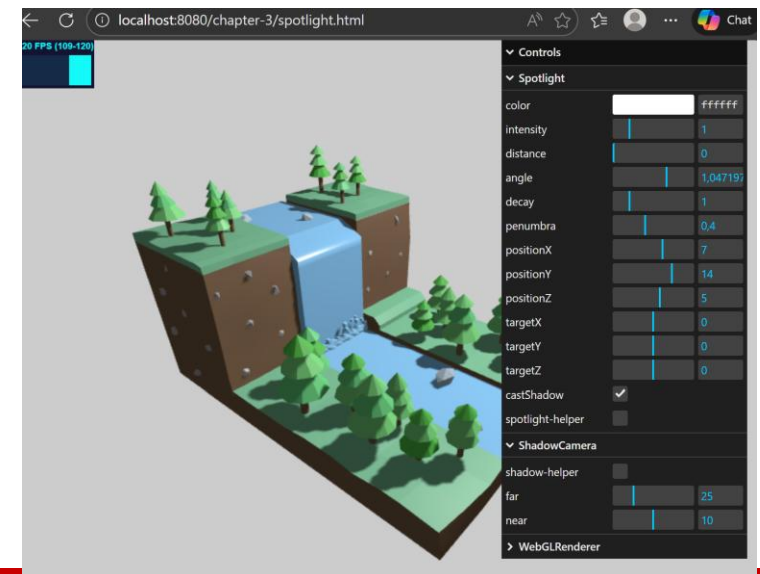
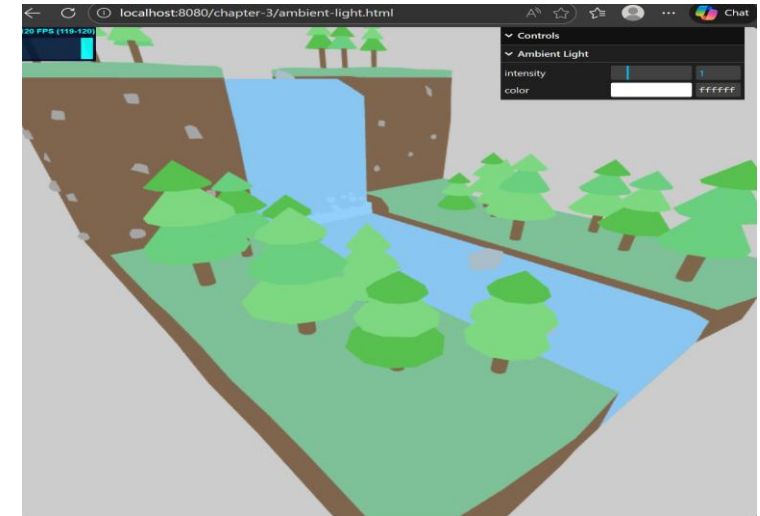
How to create a THREE.SpotLight and set the various properties to configure the light. We also explicitly set the castShadow property to true because we want shadows.

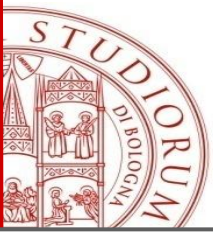
```
const spotLight = new THREE.SpotLight("#ffffff")
spotLight.penumbra = 0.4;
spotLight.position.set(10, 14, 5);
spotLight.castShadow = true;
spotLight.intensity = 1;
spotLight.shadow.camera.near = 10;
spotLight.shadow.camera.far = 25;
spotLight.shadow.mapSize.width = 2048;

spotLight.shadow.bias = -0.01;
scene.add(spotLight.target);
```



target: Locate the spotLight somewhere in scene,
By default, the target will be set to (0, 0, 0)

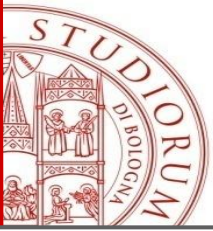




Using the THREE.Color object

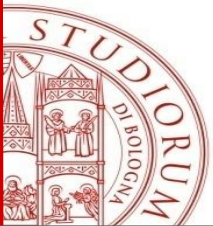
You can create a THREE.Color object in the following ways:

- **Hex value:** `new THREE.Color(0xffffff)`
will create the color based on the passed-in hex value.
- **RGB string:** `new THREE.Color("rgb(255, 0, 0)")`
- **Color name:** `new THREE.Color( 'skyblue' )`
You can use named colors as well
- **HSL string:** `new THREE.Color("hsl(0, 100%, 50%)") .`
If you like working in the HSL domain instead of the RGB domain.
- **Separate RGB values:** `new THREE.Color( 1, 0, 0)`
You can specify the individual RGB components on a scale from 0 to 1.



Examples in chapter 3:

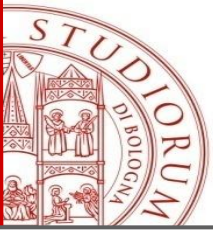
- **AmbientLight** – a globally applied light source: chapters/chapter-03/01-ambient-light.html
- **PointLight** – the light that shines in all directions: [chapters/ chapter-03/03-point-light.html](http://chapters/chapter-03/03-point-light.html)
- **SpotLight** – the light with a cone effect: chapters/chapter-03/02-spot-light.html
- **DirectionalLight** – for a far away sun-like light source: [chapters/ chapter-03/04-directional-light.html](http://chapters/chapter-03/04-directional-light.html)
- **HemisphereLight**: chapters/chapter-03/05-hemisphere-light.html
- **AreaLight**: chapters/chapter-03/06-area-light.html
- **LensFlare**: chapters/chapter-03/07-lensflares.html



Chapter 4

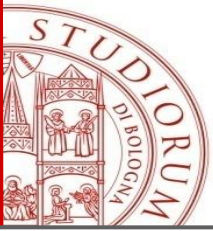
Working with Three.js Materials

we'll dive into all the materials Three.js has to offer, and you'll learn how you can use these materials to create good-looking 3D objects.



Materials in Three.js

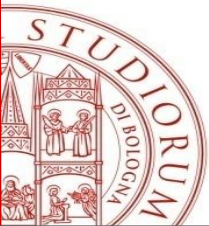
- **MeshBasicMaterial:** This is a basic material that you can use to give your geometries a simple color or show the wireframe of your geometries. This material isn't influenced by lights. ☠️
- **MeshDepthMaterial:** uses the distance from the camera to determine how to color your mesh.
- **MeshNormalMaterial:** bases the color of a face on its normal vector.
- **MeshLambertMaterial:** is used to create dull, non-shiny-looking objects. ☀️
- **MeshPhongMaterial:** can be used to create shiny objects. ☀️
- **MeshStandardMaterial:** uses physical-based rendering to render the object. With physical-based rendering, a physically correct model is used to determine how light interacts with a surface. This allows you to create more accurate and realistic-looking objects. ☀️
- **MeshPhysicalMaterial:** extension of MeshStandardMaterial that allows more control over the reflection. ☀️
- **MeshToonMaterial:** extension of MeshPhongMaterial that tries to make objects look hand-drawn.
- **ShadowMaterial:** The only thing this material does is render the shadow the mesh would receive, the mesh is rendered transparent.
- **ShaderMaterial:** This material allows you to specify shader programs to directly control how vertices are positioned and pixels are colored. Three.js provides [fragmentShader](#) and [vertexShader](#) in **GLSL**
- **LineBasicMaterial:** This is a material that can be used on the THREE.Line geometry to create colored lines.
- **LineDashMaterial:** same as LineBasicMaterial, but this material also allows you to create a dashed effect.



Common material properties

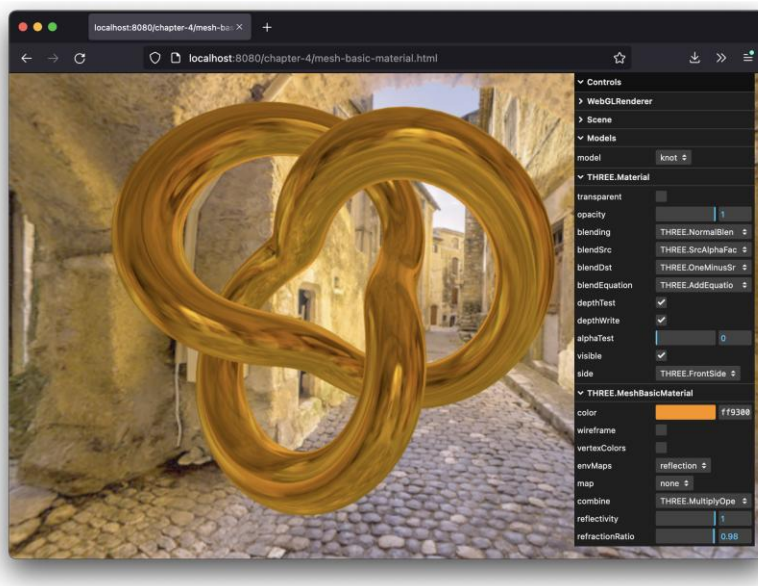
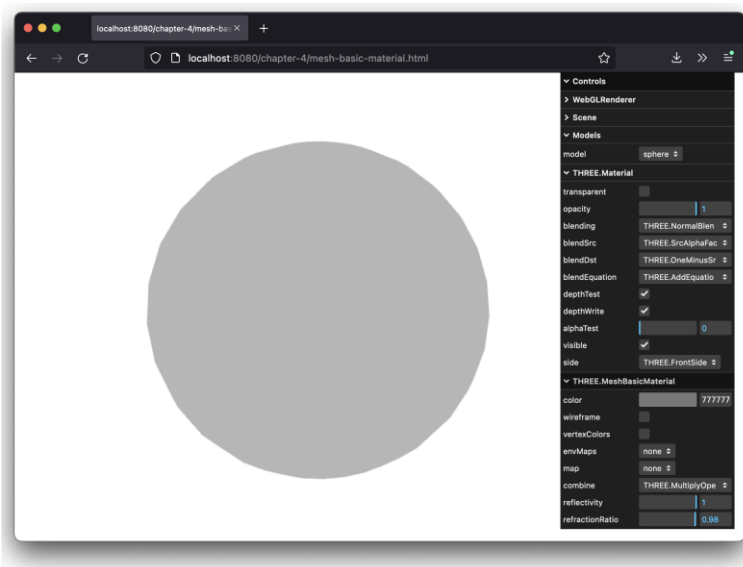
Common material properties into the following three categories:

- **Basic properties:** These are the properties you'll use most often. With these properties, you can, for instance, control the **opacity** of the object, whether it is visible, and how it is referenced (by **ID** or custom name).
- **Blending properties:** Every object has a set of blending properties. These properties define how the color of each point of the material is combined with the color behind it.
- **Advanced properties:** Several advanced properties control how the low-level WebGL context renders objects. In most cases, you won't need to deal with these properties.

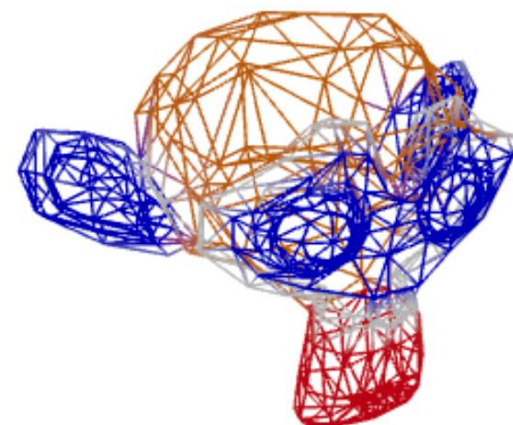


mesh-basic-material.js example

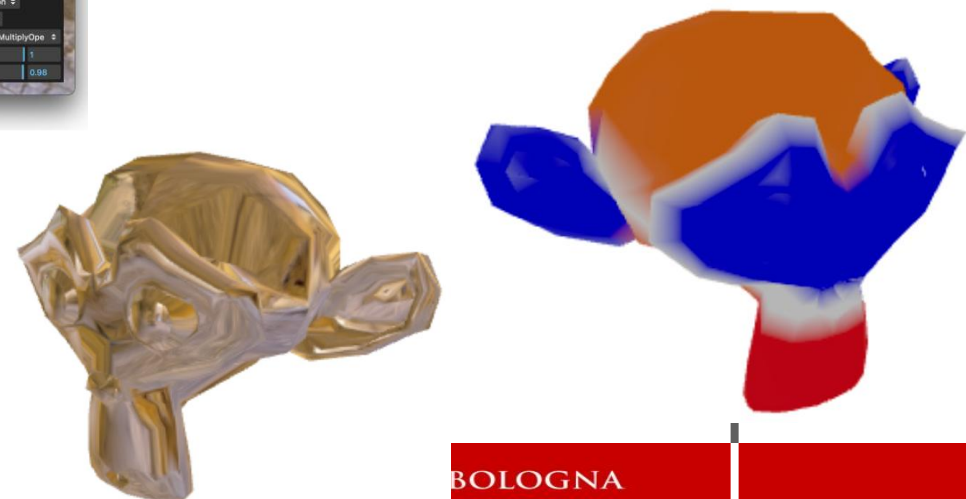
knot in env material reflection,
cube map background



Wireframe and vertex Colors



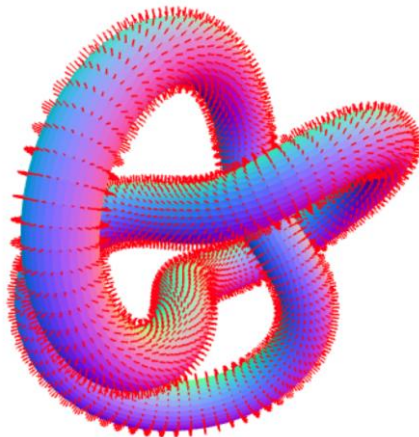
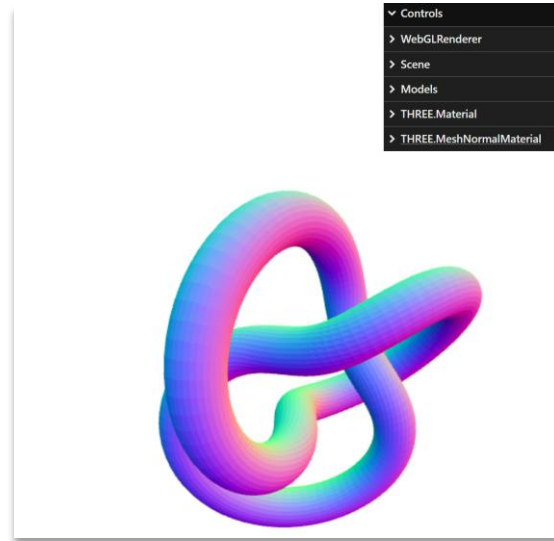
Vertex Colors



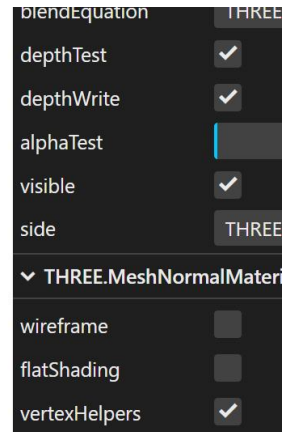
See `controls/material-controls.js`

mesh-normal-material.js

enable flatShading



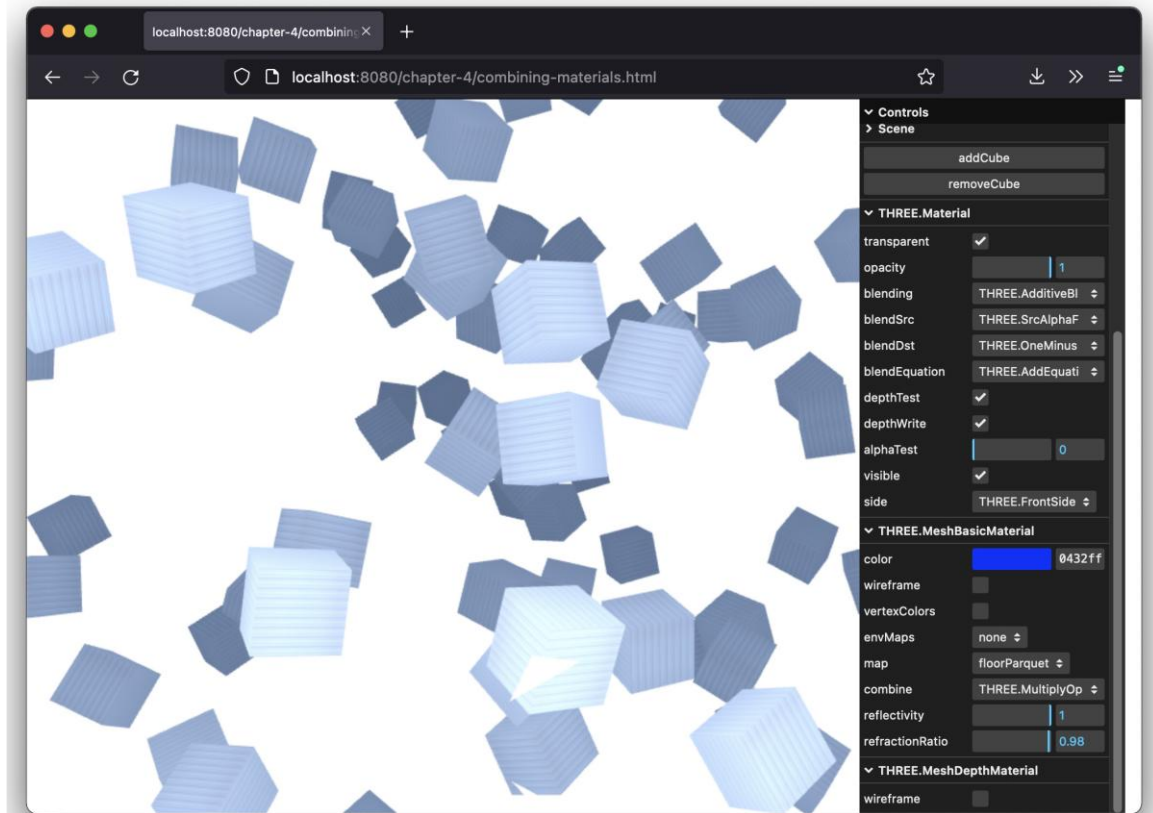
Use helper to visualize this normal



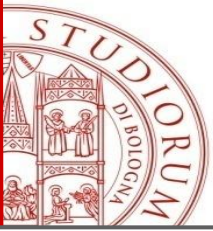
combining-materials.js

Three.js has the option to combine materials to create new effects: we get blue-colored cubes that use the brightness from **THREE.MeshDepthMaterial** and the color from **THREE.MeshBasicMaterial**.

Play around with the different blending options, and see how they affect the final color of the material.



THREE.AdditiveBlending mode :
the colors are added together, and the resulting color is shown

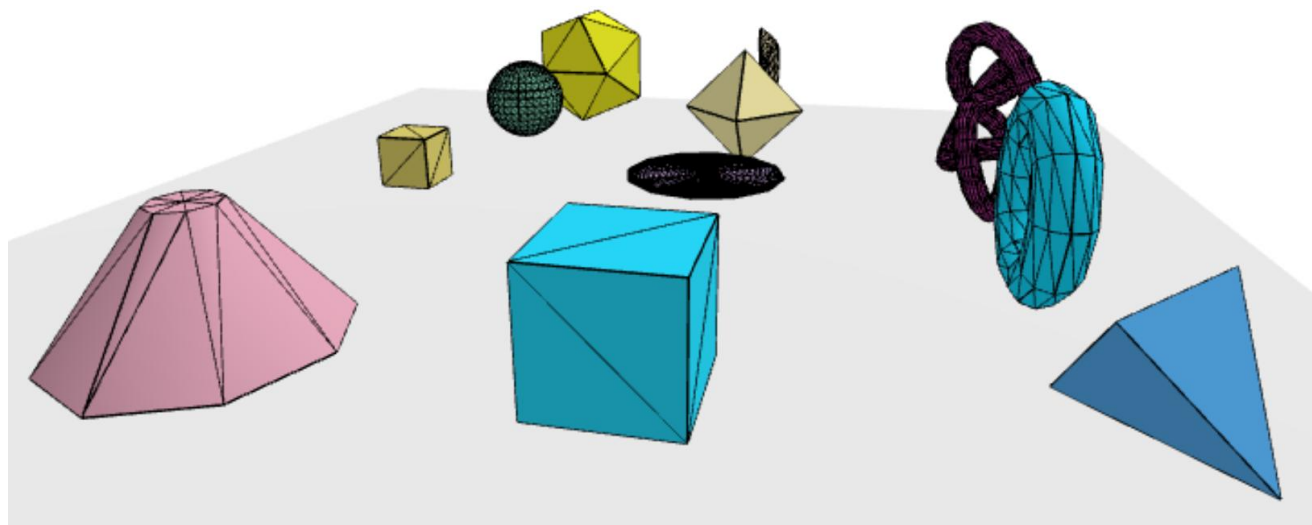


Chapter 5

In this chapter and *Chapter 6, Exploring Advanced Geometries*, we'll walk you through all the geometries that Three.js provides

2D/3D geometries

Three.js represents geometries internally as THREE.BufferGeometry

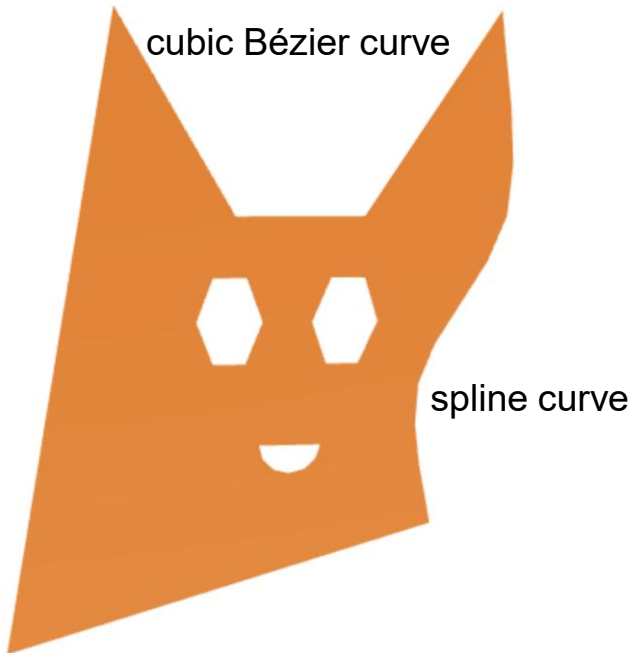


- THREE.CircleGeometry
- THREE.RingGeometry
- THREE.PlaneGeometry
- THREE.ShapeGeometry
- THREE.BoxGeometry
- THREE.SphereGeometry
- THREE.CylinderGeometry
- THREE.ConeGeometry
- THREE.TorusGeometry
- THREE.TorusKnotGeometry
- THREE.PolyhedronGeometry
- THREE.IcosahedronGeometry
- THREE.OctahedronGeometry
- THREE.TetraHedronGeometry
- THREE.DodecahedronGeometry

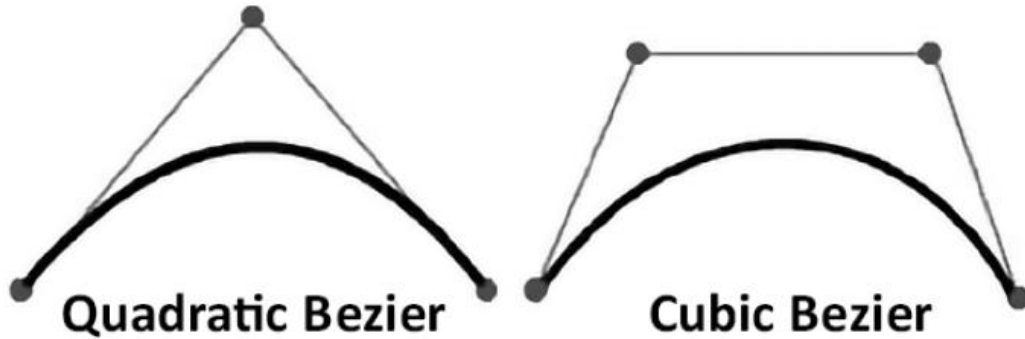
THREE.ShapeGeometry

shape-geometry.js

Create the outline of this free shape using lines, curves, and splines. After that, we punched a number of holes into this shape by using the holes property of THREE.Shape.



```
const drawShape = () => {
  // create a basic shape
  const shape = new THREE.Shape()
  // startpoint
  // straight line upwards
  shape.lineTo(10, 40)
  // the top of the figure, curve to the right
  shape.bezierCurveTo(15, 25, 25, 25, 30, 40)
  // spline back down
  shape.splineThru([new THREE.Vector2(32, 30), new
    THREE.Vector2(28, 20), new THREE.Vector2(30, 10)])
  // add 'eye' hole one
  const hole1 = new THREE.Path()
  hole1.absellipse(16, 24, 2, 3, 0, Math.PI * 2, true)
  shape.holes.push(hole1)
  // add 'eye hole 2'
  const hole2 = new THREE.Path()
  hole2.absellipse(23, 24, 2, 3, 0, Math.PI * 2, true)
  shape.holes.push(hole2)
  // add 'mouth'
  const hole3 = new THREE.Path()
  hole3.absarc(20, 16, 2, 0, Math.PI, true)
  shape.holes.push(hole3)
  return shape
}
```



- `lineTo(x, y)`

Draw a line from the current position `moveTo(x, y)` to the x- and y-coordinates that have been provided

- `quadraticCurveTo(aCPx, aCPy, x, y)`

specify one additional point (using the `aCPx` and `aCPy` arguments), and the curve ends to the specified endpoint (from the `x` and `y` arguments)

- `bezierCurveTo(aCPx1, aCPy1, aCPx2, aCPy2, x, y)`

For a cubic curve you specify two additional points to define the curve. The starting point is the current position of the path.

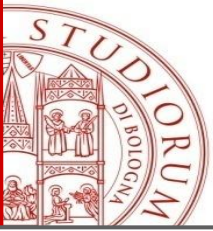
- `splineThru(pts)`

This function draws a smooth spline curve through the set of coordinates provided, (`pts`). This argument should be an array of `THREE.Vector2` objects. The starting point is the current position of the path.

It generates a **Catmull-Rom Spline**. The main feature of this curve is interpolation: the curve is forced to pass **exactly** through every coordinate you have defined in the array.

Ex: spline curve interpolating 4 points

```
const shape = new THREE.Shape();
shape.moveTo(40, 40); // Starting point
shape.splineThru([
    new THREE.Vector2(32, 30),
    new THREE.Vector2(28, 20),
    new THREE.Vector2(30, 10)
]);
```

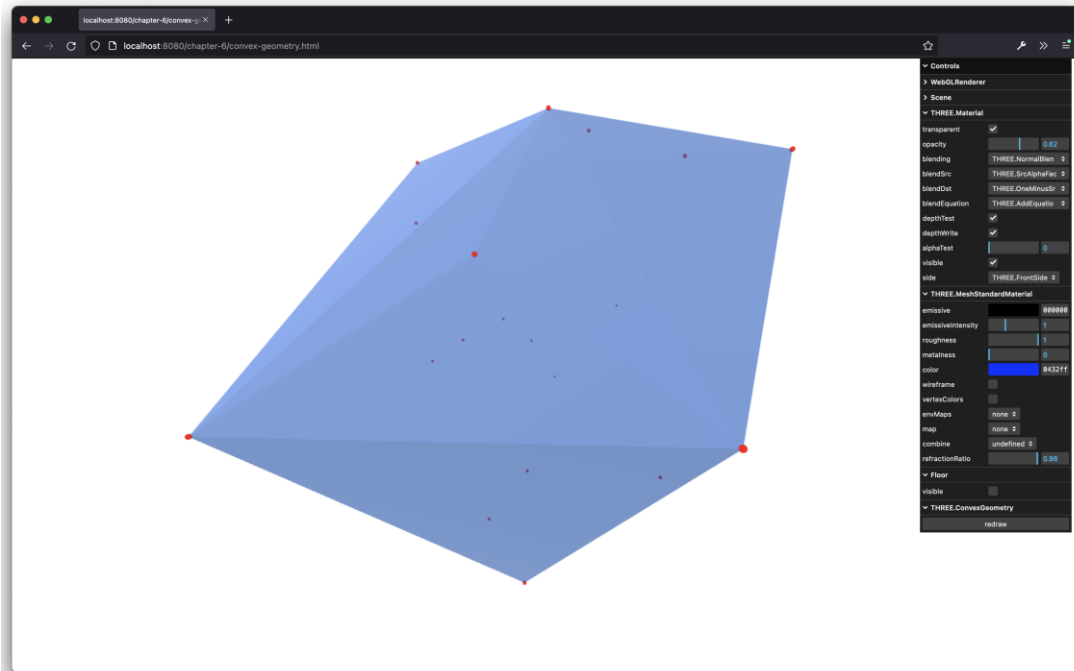
Chapter 6

Exploring Advanced Geometries

In this chapter, we'll show you these advanced geometries:

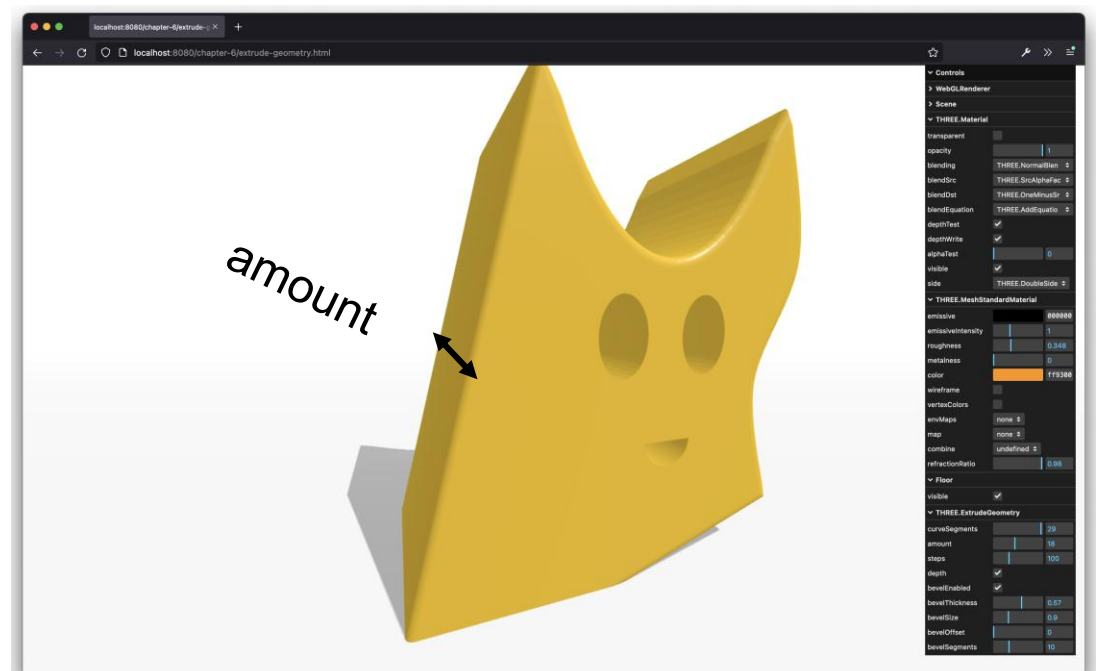
- How to use advanced geometries such as `THREE.ConvexGeometry`, `THREE.LatheGeometry`, `THREE.BoxLineGeometry`, `THREE.RoundBoxGeometry`, `THREE.TeapotGeometry`, and `THREE.TubeGeometry`.
- How to create 3D shapes from 2D shapes using `THREE.ExtrudeGeometry`. We'll create a 3D shape from a 2D SVG image, and we will extrude from 2D Three.js shapes to create novel 3D shapes.
- Three.js offers a `THREE.ParametricGeometry` object. With parametric geometry, you can create geometry with parameters you can change to effect the shape of the geometry.
- We'll also show how you can create 3D text effects using `THREE.TextGeometry`, and show you how to use the Troika library for when you want to add 2D text labels to your scene.

THREE.ConvexGeometry : we can create a convex hull from a set of points.



`convex-geometry.js`

THREE.ExtrudeGeometry : you can create a 3D object from a 2D shape

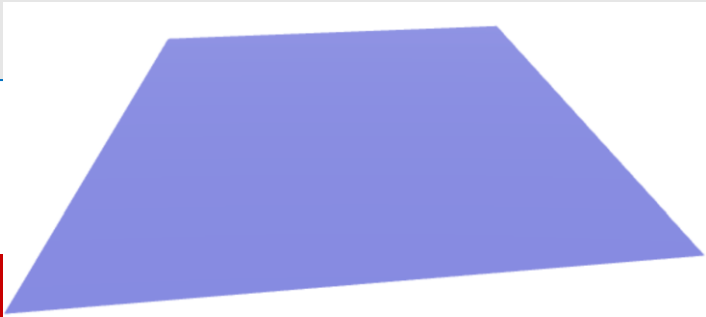


`extrude-geometry.js`

THREE.ParametricGeometry : you can create a geometry based on a parametric equation

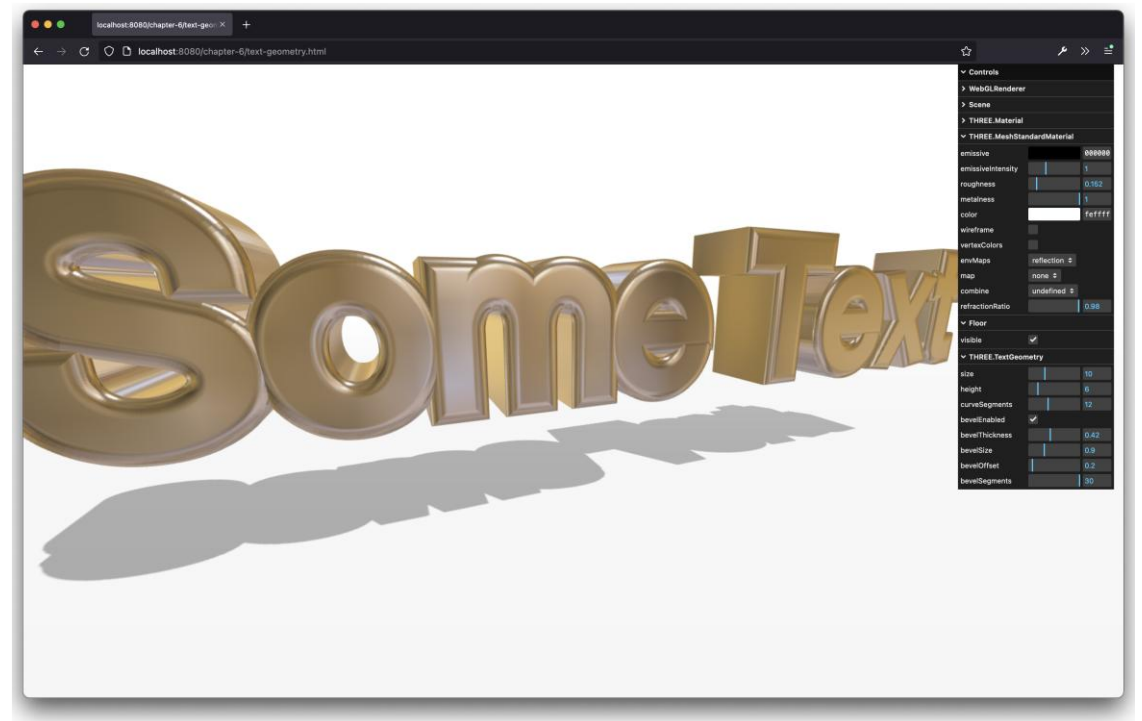
Ex: create a plane. When this is run, you'll have a basic plane with a width of **width** and a **depth** of depth

```
plane: function ( width, height ) {  
  return function ( u, v, target ) {  
    const x = u * width;  
    const y = 0;  
    const z = v * height;  
    target.set( x, y, z );  
  };  
},
```

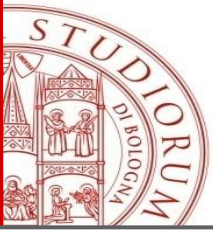


Rendering text :

```
import { FontLoader } from  
'three/examples/jsm/loaders/FontLoader'  
  
import { TextGeometry } from  
'three/examples/jsm/geometries/TextGeometry'
```



text-geometry.js



Chapter 8

Creating and Loading Advanced Meshes and Geometries

In this chapter, we'll use the following two approaches to create advanced geometries and meshes:

- Geometry grouping and merging (grouping objects together and merging multiple geometries into a single geometry)
- Loading geometries from external resources
OBJ, DAE, STL, CTM, PLY, JSON...

Geometry grouping

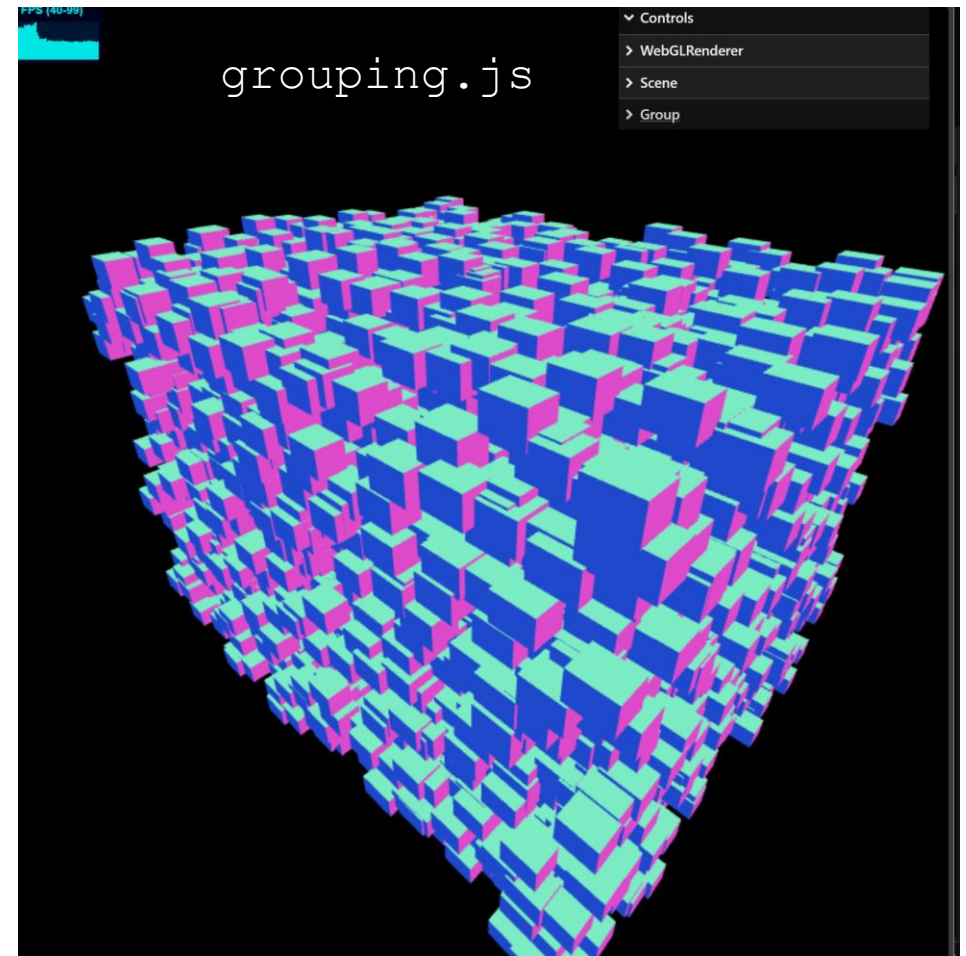
- Create a `THREE.Group` instance

Creating groups is very easy. Every mesh you create can contain child elements, which can be added using the `group.add()` function.

- The effect of adding a child object to a group is that you can move, scale, rotate, and translate the parent object, and all the child objects will also be affected.
- When using a group, you can still refer to, modify, and position the individual geometries. The only thing you need to remember is that all positions, rotations, and translations are done relative to the parent object.

Note the performance impact (FPS) of using `THREE.Group`

A large number of cubes are added to the scene as a single group

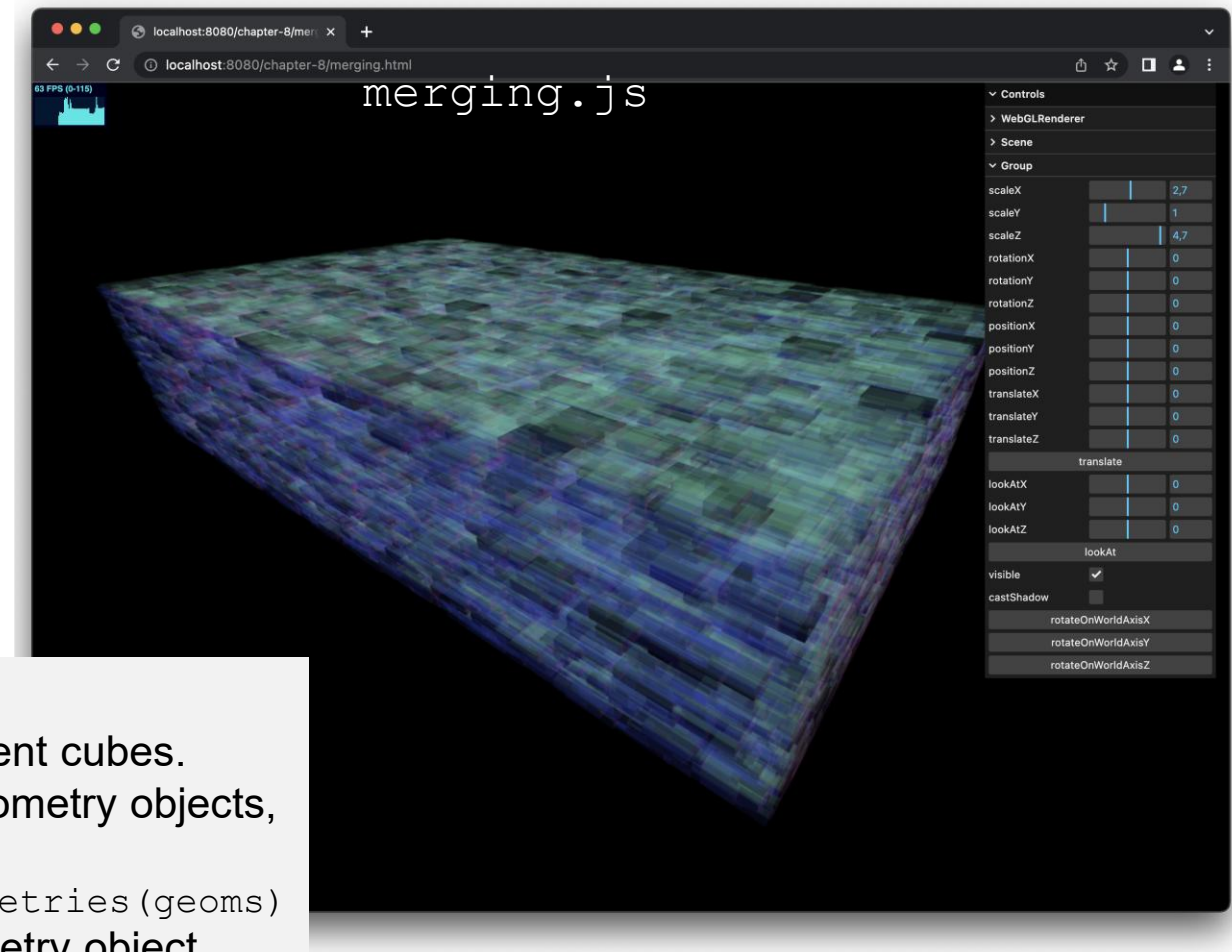


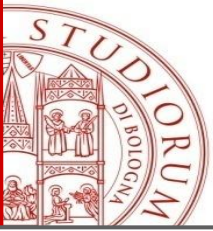
Geometry merging

Using groups allows you to easily manipulate and manage a large number of meshes. When you're dealing with a very large number of objects, however, performance will become an issue since Three.js has to treat all the children of the group individually.

With `BufferGeometryUtils.mergeBufferGeometries`, you can merge geometries together and create a combined one, so Three.js would only have to manage this single geometry.

`merging.js` :
set of randomly distributed semi-transparent cubes.
Create a large number of `THREE.BoxGeometry` objects, which we merge together using the `BufferGeometryUtils.mergeBufferGeometries` function into a single `THREE.BufferGeometry` object

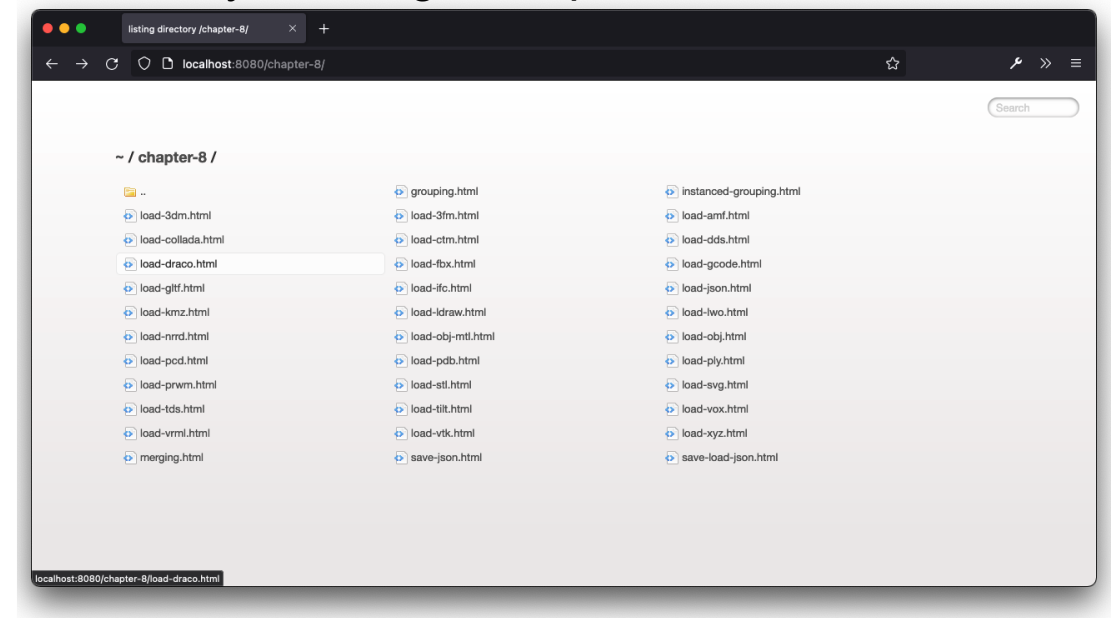




Loading geometries from external resources

- Three.js can read a large number of 3D file formats and import geometries and meshes defined in those files (**OBJ,PLY,STL,SVG,3DS,VTK,VRML,..**).
- The de facto standard for exchanging models and textures is **Graphics Language Transmission Format (GLTF)**, (.glb binary version) so if you want to load externally created models, exporting those models to GLTF format will usually give you the best results in Three.js.
- **JSON**: (export a complete scene) Three.js has its own JSON format. Even though this isn't an official format, it's very easy to use and comes in very handy when you want to reuse complex geometries or scenes.

Directory showing examples of all the loaders



Loading an OBJ and MTL model

- The OBJ and MTL formats are well supported by Three.js, so this is a good format to choose if you want to exchange 3D models.
- Three.js has two different loaders you can use. If you only want to load the geometry, you use the add-on OBJLoader.
- `load-obj-mtl.html` uses OBJLoader, together with MTLLoader, to load a model and directly assign a material.



```
import { OBJLoader } from 'three/examples/jsm/loaders/OBJLoader'  
import { MTLLoader } from 'three/examples/jsm/loaders/MTLLoader'
```

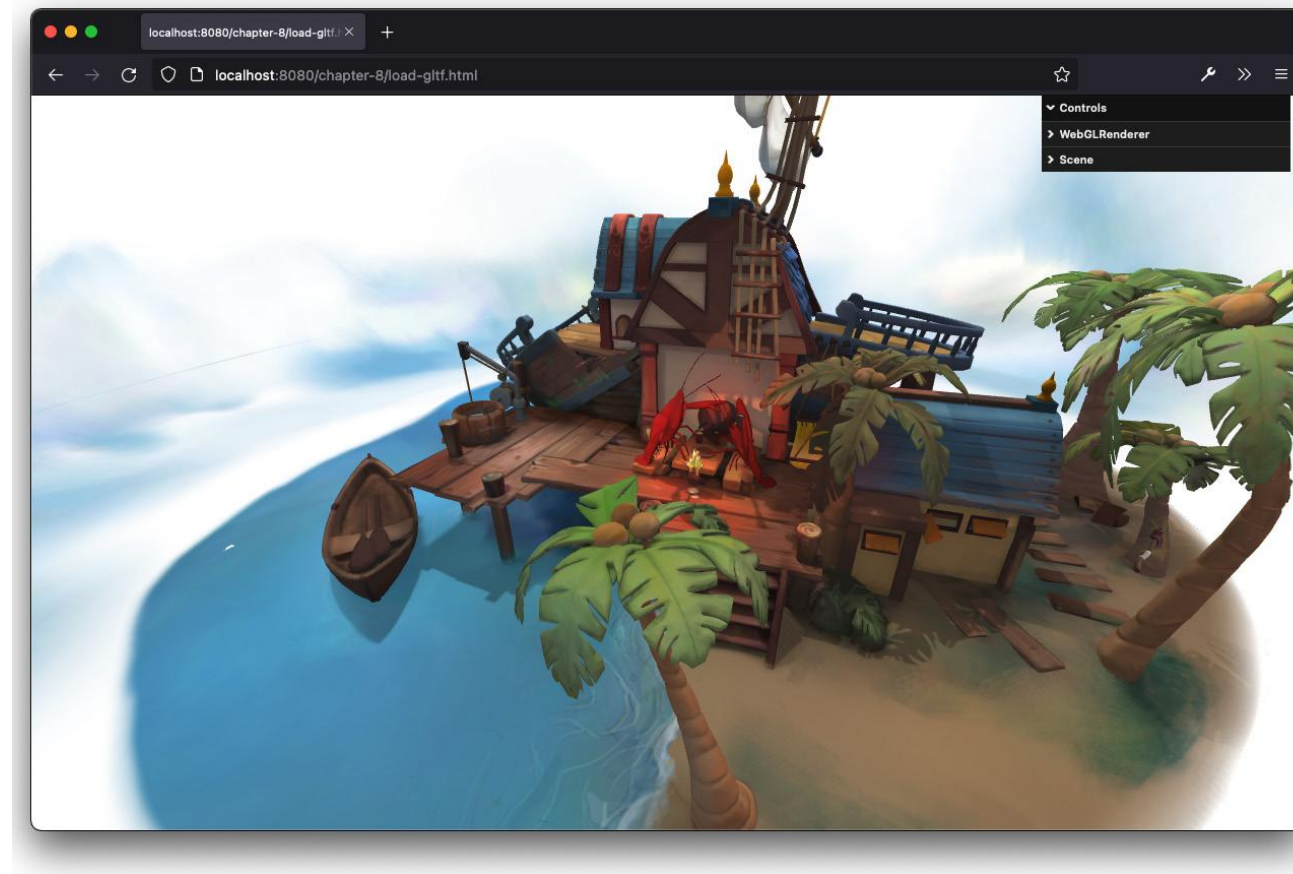
Extract mesh vertices in a sequential array : `const positions = mesh.geometry.attributes.position;`

Loading a glTF model

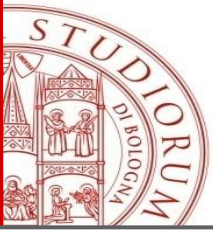
glTF come in a binary format with the **.glb** extension and a text-based format with the **.gltf** extension.

More information on this standard can be found here: [https:// www.khronos.org/glTF/](https://www.khronos.org/glTF/).

```
const loader = new GLTFLoader()|
return loader.loadAsync('/assets/models/sea_house/
scene.gltf').then((structure) => {
  structure.scene.scale.setScalar(0.2, 0.2, 0.2)
  visitChildren(structure.scene, (child) => {
    if (child.material) {
      child.material.depthWrite = true
    }
  })
  scene.add(structure.scene)
})
```



load-gltf.js



Chapter 9

Animation and Moving the Camera

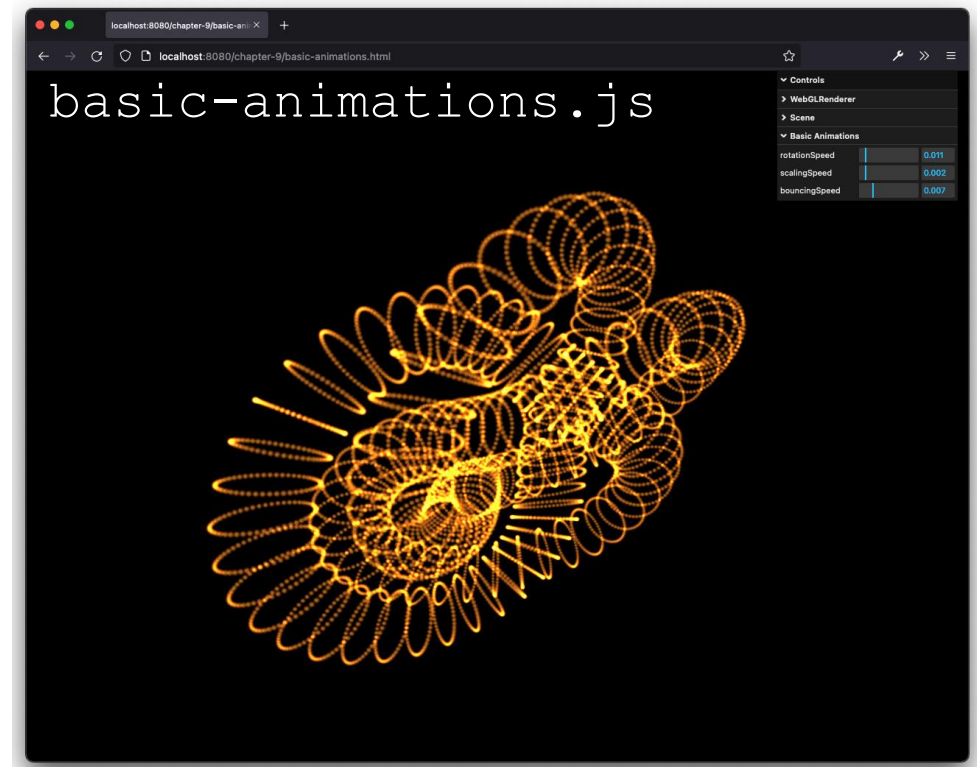
In this chapter, we're going to look in more detail at how animation is supported by Three.js.

We will look at the following four subjects:

- Basic animations
- Working with the camera
- Morphing and skeleton animation
- Creating animations using external modes

Basic animations

- we can easily animate objects by changing their rotation, scale, position, material, vertices, faces, and anything else you can imagine.
- we need to call the `render()` function once we've initialized the scene. In the `render()` function itself, we use `requestAnimationFrame` to schedule the next rendering. This way, the browser will make sure the `render()` function is called at the correct interval (usually around 60 times or 120 times a second). In the next `render()` loop, Three.js will render the changed properties.

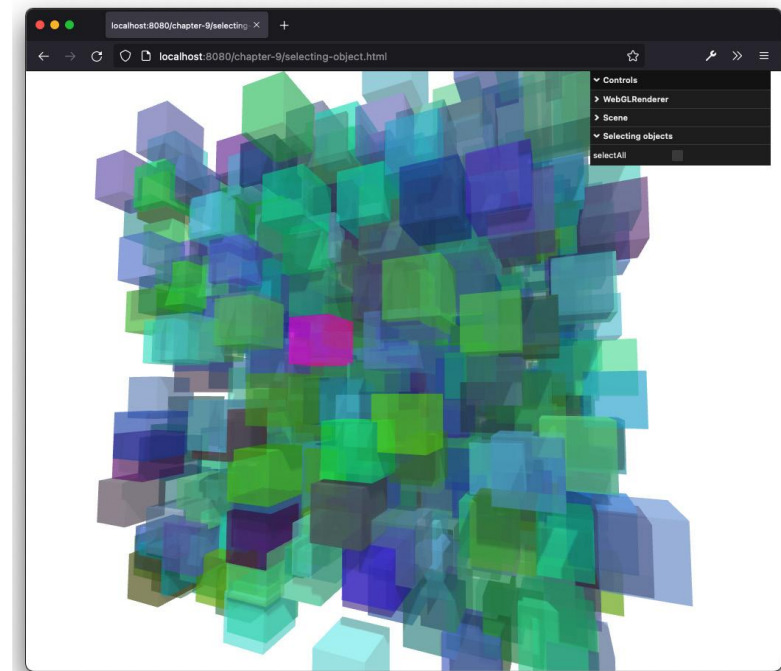


```
function animate() {  
    requestAnimationFrame(animate);  
    renderer.render(scene, camera);  
}  
animate();
```


Selecting objects

- **Select an object from a scene using the mouse**
- THREE.Raycaster. A raycaster will look at your current camera and cast a ray from the camera to your mouse's position. Based on that, it can calculate which object is hit based on the position of the mouse.
- To accomplish this, take the following steps:
 - Create an object that keeps track of where the mouse is pointing at
 - Whenever the mouse is moved, update object
 - In the render loop, use this updated information to see which Three.js object we're pointing at

Example: Randomly placed cubes that can be selected with the mouse



`selecting-objects.js`

Dragging the objects

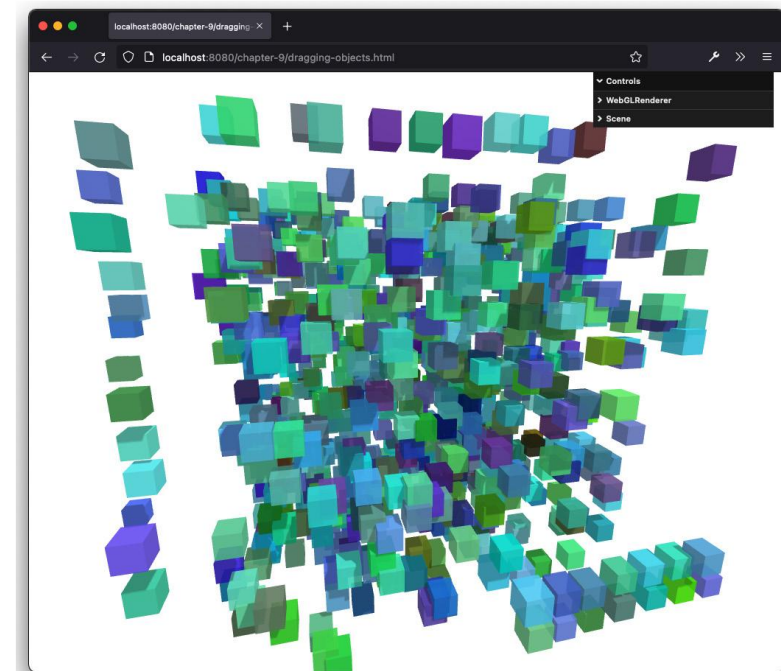
- Drag an object around the scene with the mouse
- DragControls : this handles everything and provides convenient callbacks whenever the dragging starts and stops

Example: Dragging an object around the scene using the mouse

elements that can be dragged

```
const orbit = new OrbitControls(camera, renderer.domElement)
orbit.update()
const controls = new DragControls(cubes, camera, renderer.
domElement)
controls.addEventListener('dragstart', function (event) {
  orbit.enabled = false
  event.object.material.emissive.set(0x333333)
})
controls.addEventListener('dragend', function (event) {
  orbit.enabled = true
  event.object.material.emissive.set(0x000000)
})
```

add two event listeners



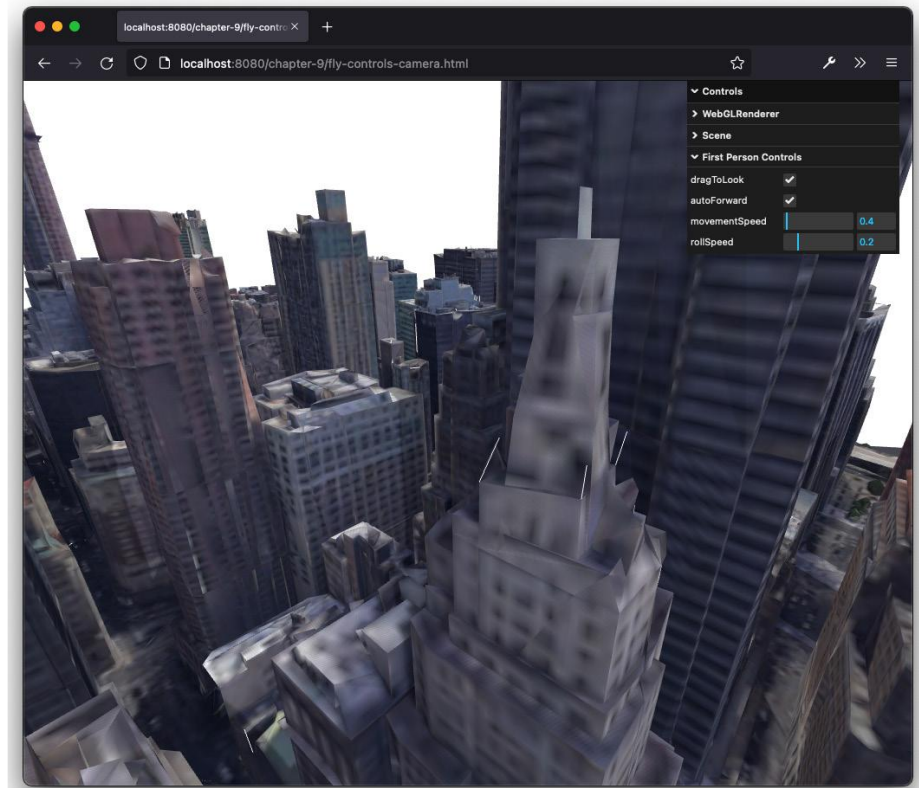
dragging-objects.js

See bootstrapMeshScene in ./util.js

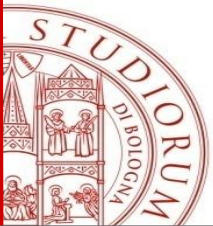
Working with the camera

- **OrbitControls:** simulates a satellite in orbit around a specific scene. This allows you to move around with the mouse and keyboard.
- **ArcballControls:** An extensive control that provides a transparent overlay that you can use to easily move the camera around.
- **FirstPersonControls:** These are controls that behave like those in first-person shooters. You can move around with the keyboard and look around with the mouse.
- **FlyControls:** These are flight simulator-like controls. You can move and steer with the keyboard and the mouse
- **TrackBallControls:** allowing you to use the mouse (or the trackball) to easily move, pan, and zoom around the scene.

```
import { FlyControls } from  
'three/examples/jsm/controls/FlyControls'
```



fly-controls-camera.js

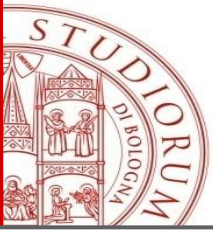


Chapter 10

Loading and Working with Textures

We'll discuss the following topics:

- Loading textures in Three.js and applying them to a mesh
- Using bump, normal, and displacement maps to apply depth and detail to a mesh
- Creating fake shadows using a lightmap and an ambient occlusion map
- Using specular, metalness, and roughness maps to set the shininess of specific parts of a mesh
- Applying an alpha map for partial transparency of an object
- Adding detailed reflections to a material using an environment map



Using textures in mesh materials

- **Loading a texture** (PNG, GIF, or JPEG images as input for a texture)

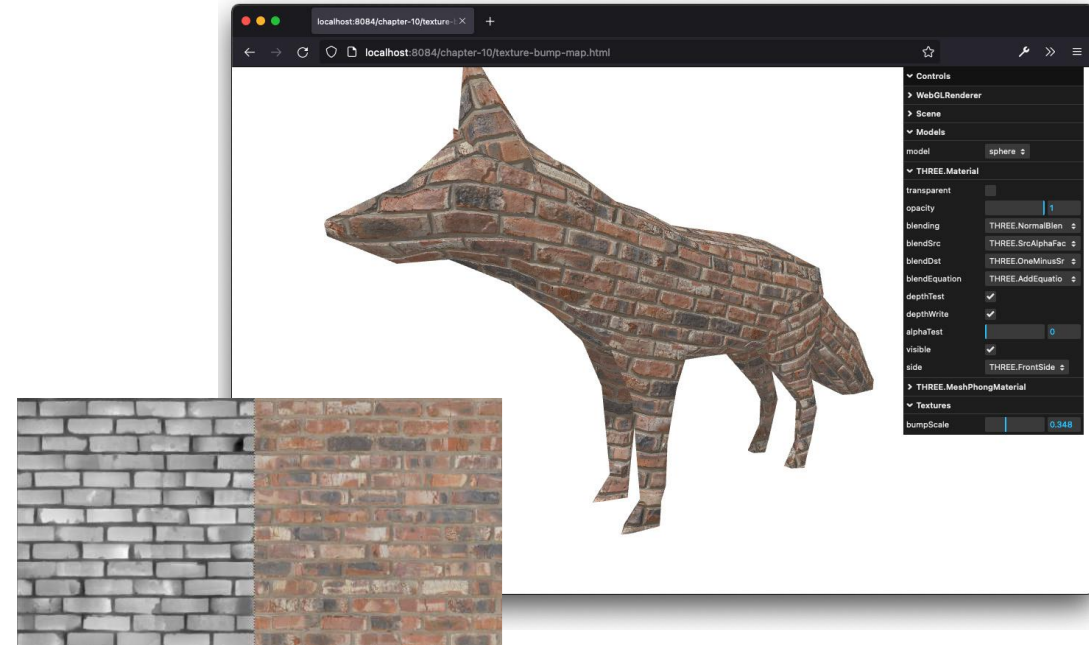
```
const textureLoader = new THREE.TextureLoader();  
const texture = textureLoader.load  
  ('/assets/textures/ground/ground_0036_color_1k.jpg')
```

- **Add it to the mesh**

```
const material = new THREE.MeshPhongMaterial({ color:  
  0xffffff })  
material.map = texture  
material.bumpMap = bumpMap
```

change the models and select textures from the menu on the right. You can also change the default material properties to see how the material, in combination with a color map, is affected by the different settings.

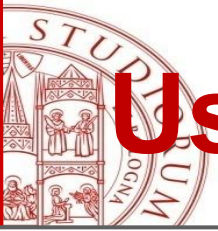
texture-basics.js



texture-bump-map.js

A bump map is used to add more depth to a material.

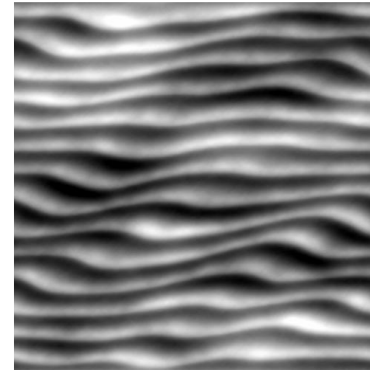
Details of UV mapping in *Chapter 13*



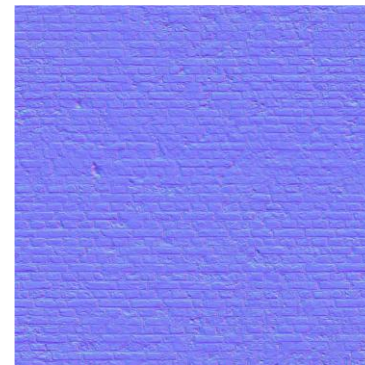
Using textures in normal, displacement map

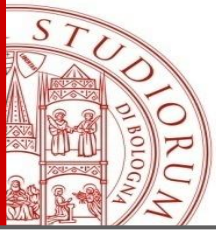
- In a **bump map**, the height (or depth, if set to a negative value) is stored, only visual effect
- In a **normal map**, the direction of the normal for each pixel is stored, only visual effect
- In a **displacement map**, the position of vertices is changed, geometry changes

- `texture-displacement.js`



- `texture-normalmap.is`



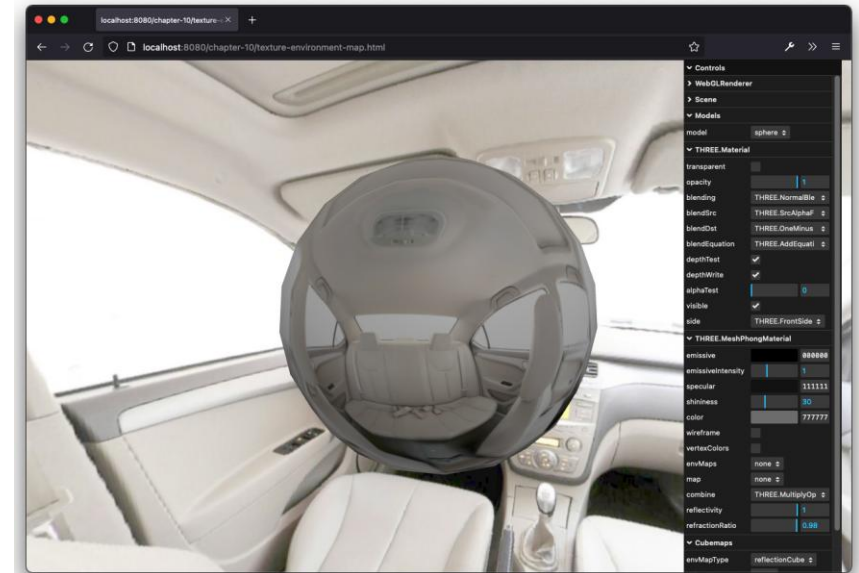


Creating fake reflections using an environment map

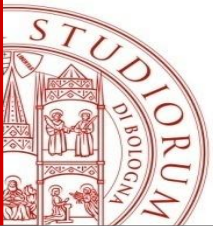
- `texture-environment-map.js`

- **creating a texture of the environment** the object is in, `CubeTexture` object.
- **Set the skybox.** We can set it as the background of the scene. We effectively create a very large box, inside of which the cameras and objects are placed, so that when we move the camera around, the background of the scene also changes correctly.
- **applying it to the specific object.**

Set the `CubeTexture` object as a texture of the `cubeMap` property of the material



```
const cubeMapCar = new THREE.CubeTextureLoader().load([
  '/assets/textures/cubemap/car/right.png',
  '/assets/textures/cubemap/car/left.png',
  '/assets/textures/cubemap/car/top.png',
  '/assets/textures/cubemap/car/bottom.png',
  '/assets/textures/cubemap/car/front.png',
  '/assets/textures/cubemap/car/back.png'
])
material.envMap = cubeMapCar
material.envMap.mapping = THREE.CubeReflectionMapping
```

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Serena Morigi

Department of Mathematics

serena.morigi@unibo.it

<https://www.unibo.it/sitoweb/serena.morigi>